



Basic Information

Experiment No: 01

Date: 16/09/2026

Title: Bank Account Management System



Objective/Aim:

To design and implement a Bank Account Management System in Java using classes and objects, demonstrating object-oriented programming concepts such as attributes, methods, object creation, deposits, withdrawals, interest calculation, and fund transfers.



System Requirements:

Hardware:

- >Standard PC/Laptop
- >Intel Core i3/i5 processor or equivalent
- >Minimum 4 GB RAM
- >Minimum 500 MB free storage

Software:

- >Operating System: Windows / Linux / macOS
- >Java Development Kit (JDK 17 or JDK 21)
- >Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- >Command Prompt / Terminal

Algorithm/Logic Description:

- 1.Start the program.
- 2.Create the Account class with account number, holder name, balance, and account type.
- 3.Define the constructor to initialize the account details.
- 4.Define methods for deposit, withdrawal, fund transfer, and displaying account details.

5. Create the SavingsAccount class by inheriting the Account class and add interestRate.
6. Define calculateInterest() to calculate and add interest to the savings account balance.
7. Create the CurrentAccount class by inheriting the Account class and add overdraftLimit.
8. Override the withdraw() method in CurrentAccount to allow withdrawal using the available overdraft limit.
9. Create objects for SavingsAccount and CurrentAccount in the main() method.
10. Display the initial details of both accounts.
11. Perform deposit, withdrawal, and interest calculation operations on the savings account.
12. Perform deposit and withdrawal operations on the current account using the overdraft facility.
13. Transfer money from the savings account to the current account.
14. Display the final details and balances of both accounts.
15. Stop the program.

UML Class Diagram:

Account:

- accountNumber : int
- accountHolderName : String
- balance : double
- accountType : String
-
- + deposit(amount) : void
- + withdraw(amount) : void

+ transfer(receiver, amount):void

+ displayAccountDetails(): void

SavingsAccount

- interestRate

: double

+calculateInterest()

:void

CurrentAccount

-overdraftLimit

: double

+withdraw(amount)

:void

Relationship:

SavingsAccount ——— inherits ——— Account

CurrentAccount ——— inherits ——— Account

Source Code:

```
package myprogs;
```

```
//Parent / Base Class
```

```
class Account {
```

```
// Attributes of Account class
```

```
int accountNumber;
```

```
String accountHolderName;
```

```
double balance;
```

```
String accountType;
```

```
// Constructor to initialize account details
```

```
Account(int accountNumber, String accountHolderName,
```

```
double balance, String accountType) {
```

```
this.accountNumber = accountNumber;
```

```
this.accountHolderName = accountHolderName;
```

```
    this.balance = balance;
```

```
    this.accountType = accountType;
```

```
}  
  
// Method to deposit money into the account  
void deposit(double amount) {  
    balance = balance + amount;  
    System.out.println("Deposited: " + amount);  
}  
  
// Method to withdraw money from the account  
void withdraw(double amount) {  
    // Check whether sufficient balance is available  
    if (amount <= balance) {  
        balance = balance - amount;  
        System.out.println("Withdrawn: " + amount);  
    } else {  
        System.out.println("Insufficient balance!");  
    }  
}  
  
// Method to transfer money from one account to another  
void transfer(Account receiver, double amount) {  
    // Check whether sufficient balance is available  
    if (amount <= balance) {  
        // Deduct money from sender account  
        balance = balance - amount;  
        // Add money to receiver account  
        receiver.balance = receiver.balance + amount;  
        System.out.println("Transferred: " + amount);  
        System.out.println("Money transferred to Account: ")
```

```

        + receiver.accountNumber);
    } else {
        System.out.println("Insufficient balance for transfer!");
    }
}

// Method to display account details
void displayAccountDetails() {
    System.out.println("Account Number   : " + accountNumber);
    System.out.println("Account Holder   : " + accountHolderName);
    System.out.println("Account Type    : " + accountType);
    System.out.println("Balance        : " + balance);
}
}

//Child Class / Derived Class
//SavingsAccount inherits properties and methods from Account
class SavingsAccount extends Account {
    // Additional attribute for Savings Account
    double interestRate;

    // Constructor of SavingsAccount
    SavingsAccount(int accountNumber, String accountHolderName,
        double balance, double interestRate) {
        // Calling the parent class constructor
        super(accountNumber, accountHolderName, balance, "Savings");
        // Assigning interest rate
        this.interestRate = interestRate;
    }
}

```

```

// Method to calculate and add interest
void calculateInterest() {
// Calculate interest using the balance and interest rate
    double interest = balance * interestRate / 100;
// Add interest to the account balance
    balance = balance + interest;
System.out.println("Interest Rate    : " + interestRate + "%");
System.out.println("Interest Added   : " + interest);
}
}

//Child Class / Derived Class
//CurrentAccount inherits properties and methods from Account
class CurrentAccount extends Account {
// Additional attribute for Current Account
    double overdraftLimit;
// Constructor of CurrentAccount
    CurrentAccount(int accountNumber, String accountHolderName,
        double balance, double overdraftLimit) {
// Calling the parent class constructor
        super(accountNumber, accountHolderName, balance, "Current");
// Assigning overdraft limit
        this.overdraftLimit = overdraftLimit;
    }
// Overriding the withdraw() method of Account class
    @Override
    void withdraw(double amount) {

```

```

// Allow withdrawal up to balance + overdraft limit
    if (amount <= balance + overdraftLimit) {
// Deduct the amount from balance
        balance = balance - amount;
System.out.println("Withdrawn: " + amount);
    } else {
// Display message if overdraft limit is exceeded
System.out.println("Withdrawal exceeds overdraft limit!");
    }
}
}

//Main Class
public class BankManagementSystem {
    public static void main(String[] args) {
// Creating an object of SavingsAccount
        SavingsAccount savings =new SavingsAccount(101, "Sai", 20000, 5);
// Creating an object of CurrentAccount
        CurrentAccount current =new CurrentAccount(102, "Pujitha", 15000,
5000);
// Display initial account details
        System.out.println("===== INITIAL ACCOUNT DETAILS =====");
// Display Savings Account details
System.out.println("\nSavings Account");
savings.displayAccountDetails();
// Display Current Account details
System.out.println("\nCurrent Account");
current.displayAccountDetails();

```

```
// Performing Savings Account transactions

System.out.println("\n===== SAVINGS ACCOUNT TRANSACTIONS
=====");

// Deposit money into Savings Account
savings.deposit(5000);

// Withdraw money from Savings Account
savings.withdraw(3000);

// Calculate interest on Savings Account
savings.calculateInterest();

// Performing Current Account transactions

System.out.println("\n===== CURRENT ACCOUNT TRANSACTIONS
=====");

// Deposit money into Current Account
current.deposit(5000);

// Withdraw money using overdraft facility
current.withdraw(22000);

// Transfer money from Savings Account to Current Account
System.out.println("\n===== FUND TRANSFER =====");
savings.transfer(current, 4000);

// Display final account details
System.out.println("\n===== FINAL ACCOUNT DETAILS =====");

// Display final Savings Account details
System.out.println("\nSavings Account");
savings.displayAccountDetails();

// Display final Current Account details
System.out.println("\nCurrent Account");
current.displayAccountDetails();
```

```
}  
}
```

Compilation&Execution Commands:

Save the source code exactly as:

BankAccountManagementSystem.java

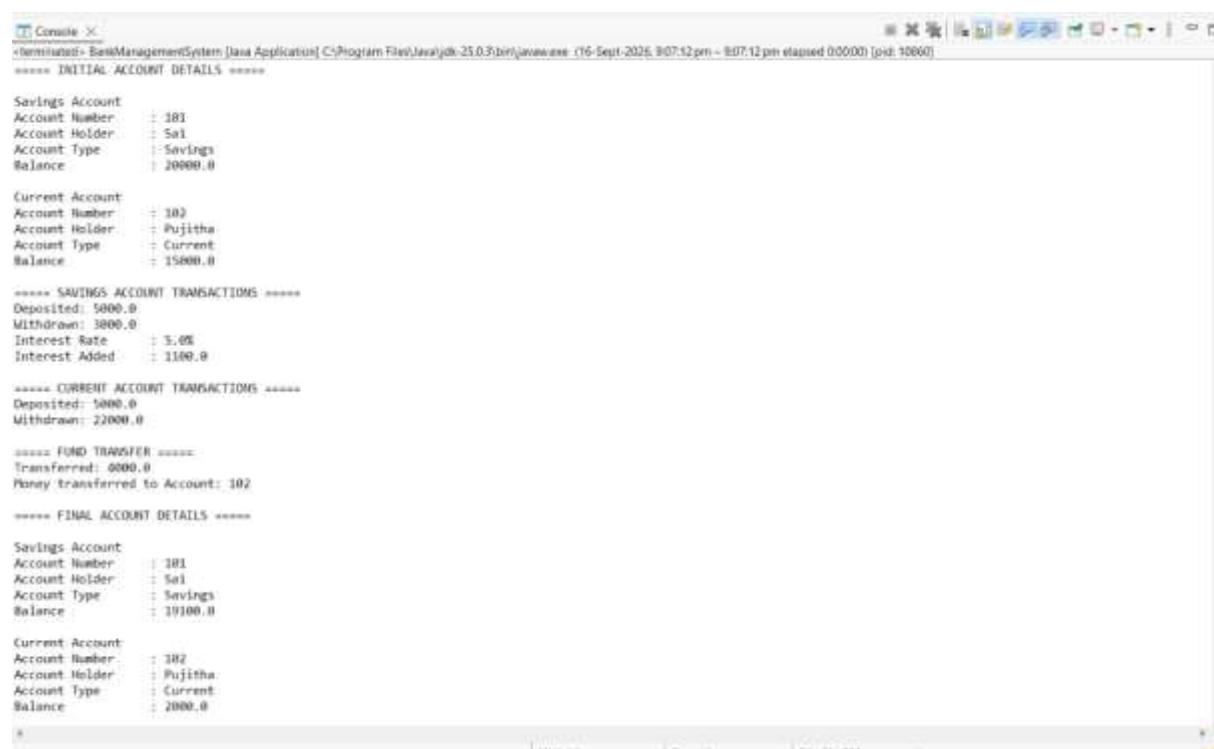
Compile the program:

```
javac BankAccountManagementSystem.java
```

Run the compiled program:

```
java BankAccountManagementSystem
```

Output Screenshot:



The screenshot shows a Java application window titled "BankManagementSystem [Java Application]". The window displays the following output:

```
<terminated> BankManagementSystem [Java Application] C:\Program Files\Java\jdk-25.0.3\bin\java.exe -c16-Sept-2026, 9:07:12 pm - 9:07:12 pm elapsed 0:00:00 [pid: 10860]  
===== INITIAL ACCOUNT DETAILS =====  
  
Savings Account  
Account Number : 101  
Account Holder : Sai  
Account Type : Savings  
Balance : 20000.0  
  
Current Account  
Account Number : 102  
Account Holder : Pujitha  
Account Type : Current  
Balance : 15000.0  
  
===== SAVINGS ACCOUNT TRANSACTIONS =====  
Deposited: 5000.0  
Withdrawn: 3000.0  
Interest Rate : 3.0%  
Interest Added : 1500.0  
  
===== CURRENT ACCOUNT TRANSACTIONS =====  
Deposited: 5000.0  
Withdrawn: 22000.0  
  
===== FUND TRANSFER =====  
Transferred: 6000.0  
Money transferred to Account: 102  
  
===== FINAL ACCOUNT DETAILS =====  
  
Savings Account  
Account Number : 101  
Account Holder : Sai  
Account Type : Savings  
Balance : 19100.0  
  
Current Account  
Account Number : 102  
Account Holder : Pujitha  
Account Type : Current  
Balance : 2000.0
```

Result:

The Bank Account Management System was successfully implemented in Java using classes and objects. The program demonstrates the creation of

SavingsAccount and CurrentAccount objects and performs deposits, withdrawals, interest calculation, and fund transfers successfully.